

Guion de la demo

Jev decide, Claude trabaja

Qué pasa en cada demo, qué comando o skill se usa, qué se le pregunta a Jev y dónde vive cada cosa. Para leer antes de la sesión.

Recurso	Dónde
Repo	github.com/LIDR-academy/jev-dev-demo (carpeta local: eventojev/jev-dev-demo)
Código de la demo 2	github.com/LIDR-academy/medusa (carpeta local: eventojev/medusa-fork)
Jira	lidr-demo.atlassian.net · proyecto KAN · tickets KAN-221 a KAN-250
PRs de la demo 3	github.com/LIDR-academy/medusa/pulls (20 PRs abiertas)
Duración	30 minutos: demo 1 (10) · demo 2 (8) · demo 3 (8) · cierre (4)
Estado del ensayo	7 de octubre de 2026, todo con servicios reales (Jev, Claude por CLI, Jira, GitHub)

El patrón en una línea. Estado → pregunta tipada → Jev → probabilidad → **tu código decide** → acción. Jev no escribe ni razona: elige entre opciones que tú definiste y dice con qué probabilidad. Claude solo trabaja donde Jev dice que hace falta, con el modelo que hace falta.

1. Panorama

Son tres demos sobre el mismo patrón. En las tres, Jev responde preguntas cerradas en milisegundos y por fracciones de centavo; unos umbrales escritos en código deciden qué hacer con esas probabilidades; y Claude solo entra cuando hace falta pensar.

Demo	Pregunta de negocio	Qué decide Jev	Qué hace Claude	Comando o skill clave
1 · Triage min 0–10	¿Qué tipo, prioridad y equipo tiene cada ticket?	4 preguntas por ticket: tipo, prioridad, equipo, si necesita humano	Crea el issue desde una queja (MCP de Atlassian)	<code>/triage</code> <code>npm run batch -- --no-create</code>
2 · Pre-vuelo min 10–18	Antes de tocar este código, ¿dónde están los problemas?	6 preguntas por función de Medusa real	Explica solo las funciones marcadas	<code>npm run scan -- --ticket=T03 --explain /pre-vuelo T03</code>
3 · Router min 18–26	¿Con qué modelo revisar cada PR?	Complejidad y si toca dinero → reglas, Haiku o Sonnet	Revisa cada PR con el modelo elegido	<code>npm run review -- --mode=jev ...</code> <code>npm run compare</code>

Dónde está cada cosa

Qué	Dónde
La demo completa (código, comandos, skills)	<code>eventojev/jev-dev-demo</code> → GitHub <code>LIDR-academy/jev-dev-demo</code>
Código real que escanea la demo 2	<code>eventojev/medusa-fork</code> (412 MB) → <code>LIDR-academy/medusa</code> , commit d436d2fb
Las 20 PRs de la demo 3	PRs abiertas en <code>LIDR-academy/medusa</code> ; su copia local en <code>data/prs-fork.json</code>
Llaves (Jev, Anthropic, Jira)	<code>jev-dev-demo/.env</code> (nunca se sube; nunca se muestra en pantalla)
Preguntas a Jev de la demo 1	<code>jira-triage/classify.ts</code>
Preguntas a Jev de la demo 2	<code>code-health/scan.ts</code>
Preguntas a Jev de la demo 3	<code>llm-router/router.ts</code>
Los umbrales (lo que "decide tu código")	<code>shared/thresholds.ts</code> y <code>SCAN_THRESHOLD</code> en <code>.env</code>
Prompts de Claude	<code>code-health/explain.ts</code> (demo 2) · <code>llm-router/llm.ts</code> (demo 3)
Cliente de Jev / de Claude	<code>shared/jev.ts</code> · <code>shared/claude.ts</code> (motor <code>CLAUDE_ENGINE=cli</code>)
Skills de Claude Code	<code>.claude/skills/</code> : <code>triage</code> , <code>pre-vuelo</code> , <code>costos</code>
MCP de Atlassian	<code>.mcp.json</code> (se autentica con <code>/mcp</code> → <code>atlassian</code>)
Precios	<code>llm-router/prices.json</code>
Resultados de cada corrida	<code>results/</code> (no se sube) · respaldo en <code>results-backup/</code>
Guion corto y referencia	<code>README.md</code> (tabla "El día de la demo") · <code>GUIA-CLAUDE-CODE.md</code>

Cómo se le habla a Jev

Jev (TypeSafe AI) es un modelo *System One*: no genera texto, responde **preguntas tipadas** con probabilidades calibradas. Cada llamada es un `POST https://api.typesafe.ai/v1/systemone` con tres campos: `model` (`jev-1.13.0`), `state` (los datos: un ticket, una función o una PR) y `questions`. Cobra solo los tokens de entrada: \$0.042 por millón.

Tipo	Qué devuelve	Ejemplo en la demo
choice	Una opción de una lista cerrada, su confianza y la probabilidad de cada opción	Prioridad: critica / alta / media / baja
noul	Una probabilidad de 0 a 1 de que algo sea verdad (sí/no)	¿Necesita a un humano? ¿Se traga errores?
score	Un valor en una escala ordenada (0 a N) con su leyenda	Severidad de 0 (limpia) a 3 (cobros incorrectos)

Anatomía de un prompt a Jev

Con un LLM escribes un prompt de texto libre y lees texto libre. Con Jev el "prompt" es estructurado y tiene tres partes:

- **state**: los datos sobre los que se decide (un ticket, una función o una PR) como JSON. Solo hechos, nunca instrucciones.
- **instructions**: la pregunta, en una frase. Debe decir qué juzgar y, si hace falta, qué *no* juzgar ("el impacto de negocio, no lo alarmante del texto").
- **criteria**: las opciones posibles, cada una con una descripción concreta. Es la parte que más pesa: Jev compara el `state` contra cada descripción. Las descripciones vagas dan probabilidades repartidas y confianzas bajas; con ejemplos concretos ("cobros duplicados", "un navegador") la confianza sube.

Cómo se lee la respuesta:

- **choice** devuelve `choice` (la opción ganadora), `probabilities` (una por opción, suman 1) y `confidence` (qué tan segura está Jev de la ganadora). Cuando dos opciones están parejas la confianza baja aunque una gane: en la demo 3, PR-15 dio alta 37 % frente a media 35 % y confianza 6 %.
- **noul** devuelve un solo número de 0 a 1: la probabilidad de que la afirmación sea cierta (0.97 = "casi seguro que sí").
- **score** devuelve `score`, el promedio ponderado de la escala: con probabilidades 2 % / 2 % / 59 % / 37 % para 0, 1, 2 y 3, el score es 2.3.

La regla de oro de la demo: Jev nunca "decide la acción". Devuelve probabilidades; tu código, con umbrales escritos y versionados, decide qué hacer. Si Jev clasifica mal, se afinan las descripciones de los *criteria*, no los umbrales.

En la sección de cada demo está el texto exacto de cada pregunta y de cada opción. Las opciones se llaman *criteria*: son lo único que se afina cuando Jev clasifica mal. Los umbrales no se tocan para "hacer que salga".

2. Antes de la sesión

Ya está todo limpio desde el ensayo del 7 de octubre. Si después ensayas otra vez, repite lo que dice "Si ensayaste".

Checklist del día

1. Abre una terminal y entra a la demo: `cd ~/Documents/lidr/eventojev/jev-dev-demo && claude`. Los skills `/trriage`, `/pre-vuelo` y `/costos` solo existen si Claude Code se abre **dentro de esta carpeta**.
2. En Claude Code: `/mcp` → **atlassian** → autentícate. Sin esto, `/trriage` no puede crear el issue.
3. Comprueba que todo responde: `npm test` (14 en verde). Si quieres probar Jev sin tocar nada: `npm run scan -- --ticket=T03` y luego borra `results/code-health.json`.
4. Comprueba que `results/` solo tiene los tres "antes" ya calculados: `trriage-direct.json` (demo 1), `code-health-direct.json` (demo 2) y `direct.json` (demo 3).
5. Abre en el navegador, en pestañas: el tablero de KAN en Jira, el ticket **KAN-223** (T03, para el minuto 10), la lista de PRs del fork y VS Code con la carpeta `jev-dev-demo`.
6. En VS Code deja a mano: `shared/thresholds.ts` (min 2), `../medusa-fork/packages/modules/payment/src/services/payment-module.ts` (min 10) y `llm-router/router.ts` (min 19).
7. Ten el celular con hotspot listo por si falla la red del lugar.

Estado esperado al empezar

Dónde	Cómo debe estar
Jira · KAN	30 tickets con etiqueta batch , de KAN-221 a KAN-250 : 8 en Finalizada ("sprint anterior") y 22 sin clasificar. Ninguno con etiqueta <code>jev</code> ni comentario de Jev.
GitHub · LIDR-academy/medusa	20 PRs abiertas, sin comentarios del router.
jev-dev-demo/results/	Solo los tres "antes": <code>trriage-direct.json</code> , <code>code-health-direct.json</code> y <code>direct.json</code> .
medusa-fork	En el commit <code>d436d2fb</code> . Si actualizas el fork, vuelve a correr el <code>pre-vuelo</code> y confirma la marca de <code>roundToCurrencyPrecision</code> .
.env	<code>CLAUDE_ENGINE=cli</code> (Claude con tu sesión de Claude Code; la API de Anthropic no tiene saldo) y <code>JIRA_PROJECT_KEY=KAN</code> .

Keys de Jira

Jira no reutiliza números: cada `reset` + `seed` crea keys nuevas. Estas son las del 7 de octubre. La key es siempre **KAN-(220 + número del ticket)**.

Ticket	Key	Dónde se usa
T01 a T30 (los 30 sembrados)	KAN-221 a KAN-250	Tablero del minuto 0 y <code>batch</code> del minuto 6
T03 "Reembolso parcial se registra como total"	KAN-223	Demo 2, minuto 10
T28, T29, T30 (los 3 ambiguos)	KAN-248, 249, 250	Filtro <code>labels = escalado</code> , minuto 8
T20 a T27 (los 8 en Finalizada)	KAN-240 a KAN-247	Contexto del tablero

Ticket	Key	Dónde se usa
El que crea /triage en vivo	KAN-251	Minutos 3 y 5. Es el siguiente número libre: si creas otro issue antes de la sesión, será otro.

Si ensayaste, o después de la sesión: cómo dejarlo limpio

#	Dónde	Qué haces / escribes	Qué debe aparecer	Qué dices
1	Terminal	<code>npm run reset</code>	Borra los issues de la demo en KAN (etiquetas jev, batch, revisar, escalado) y vacía results/, salvo los tres "antes" .	
2	Terminal	<code>npm run seed -- --done=8</code>	30 tickets nuevos, 8 a Finalizada. Con keys nuevas: actualiza la tabla "Keys de Jira".	
3	Claude Code	Pide: "borra los comentarios del router en las PRs de LIDR-academy/medusa".	0 comentarios en las 20 PRs.	
4	Terminal	<code>Solo si los borraste con npm run reset -- --baselines: npm run batch -- --mode=direct npm run scan -- --ticket=T03 - -mode=direct npm run review -- -- mode=direct --data=data/prs- fork.json</code>	Los tres "antes" de nuevo (~2 min en total). El de la demo 1 lee los 22 issues nuevos de KAN.	

Ojo: si en el ensayo usaste `--comment`, los comentarios del router quedan en las 20 PRs del fork. Hay que borrarlos antes de la sesión (pídeselo a Claude Code). `npm run reset` solo toca Jira y results/.

3. Minuto a minuto

Min	Acción / comando	Qué se ve y qué dices
0	Jira: tablero de KAN	30 tickets (KAN-221 a KAN-250), 8 en Finalizada, 22 sin clasificar. "Esto ve un manager cada lunes."
2	VS Code: shared/thresholds.ts	Los umbrales son código tuyo, no del modelo.
3	Claude Code: /triage Pagué con débito y me llegaron dos cargos por el mismo pedido	Claude crea KAN-251 con el MCP; Jev lo clasifica: bug / crítica / pagos → apply.
5	Abrir KAN-251 en Jira	Comentario de Jev con las tres decisiones y su porcentaje.
6	npm run batch -- --no-create	22 tickets clasificados en ~2 s, tabla y costo de una fracción de centavo.
8	Filtro en Jira labels = escalado	Los 3 ambiguos (KAN-248, 249, 250) escalados a una persona. (Los marcados para revisión llevan otra etiqueta: revisar.)
10	Demo 2. KAN-223 (T03) en Jira + payment-module.ts de Medusa en VS Code	"Esto es Medusa real, 1,468 líneas. Antes de tocarlo, ¿dónde están los problemas?"
11	npm run scan -- --ticket=T03	58 funciones en ~2 s: 1 o 2 marcadas, el resto limpias.
13	npm run scan -- --ticket=T03 --explain (o /pre-vuelo T03)	Claude explica solo lo marcado: el catch vacío de roundToCurrencyPrecision. Reutiliza lo que Jev dijo en el min 11: mismos números, ~7 s.
15	El node -e de JPY (demo 2)	JPY → TypeError: el bug que el catch esconde, en vivo.
16	Señalas las funciones limpias	"Estas no gastaron un token."
17	npm run costs -- --demo=1,2	Antes y después de las demos 1 y 2: triage \$0.078 → \$0.0009; pre-vuelo \$0.235 → \$0.022. Sin --demo saldría ya el direct de la demo 3.
18	Demo 3. PRs del fork en el navegador + npm run costs	20 PRs reales. La fila direct: todo a Sonnet, tantos tokens, tanto costo.
19	VS Code: llm-router/router.ts	Una pregunta, tres caminos.
20	npm run review -- --mode=jev --data=data/prs-fork.json --comment	Cada PR con su ruta; los comentarios aparecen en GitHub.
23	npm run compare	Tabla lado a lado. Silencio. Luego el número.
25	npm run show-review -- PR-12	La PR de pagos recibió la misma revisión en ambos modos.
26	Cierre. npm run review -- --mode=jev --data=data/prs-fast-jev-compaction.json --out=results/cierre.json --concurrency=6	Un plugin de Claude Code que usa Jev, revisado por un router que usa Jev. ~30 s: habla mientras corre.
28	Patrón en una diapositiva	"En la demo 2 nadie sabía que ese bug estaba en Medusa; Jev señaló una función de 58 y ahí estaba." · "Jev no escribe, Jev elige."
29	npm run costs final (o /costos)	Lo que costó toda la demo, con el cierre en su propia fila.

DEMO 1

Triage del backlog de Jira

Minutos 0–10 · Qué tipo, prioridad y equipo tiene cada ticket, y cuáles necesitan a una persona

Qué se cuenta

Un backlog desordenado: 30 tickets de una tienda en línea. Ocho ya están resueltos (dan sensación de proyecto vivo) y 22 llegan sin tipo, prioridad ni equipo. Jev los clasifica en paralelo en un par de segundos; tu código decide si aplica directo, aplica y marca para revisión, o escala a una persona. Antes, en vivo, Claude Code crea un ticket nuevo a partir de una queja de cliente y Jev lo clasifica al instante.

Secuencia de pasos

#	Dónde	Qué haces / escribes	Qué debe aparecer	Qué dices
1	Jira	Abre el tablero de KAN.	30 tickets (KAN-221 a KAN-250): 22 en Tareas por hacer, 8 en Finalizada. Ninguno con tipo ni prioridad.	"Esto ve un manager cada lunes: 22 tickets sin tipo, prioridad ni dueño."
2	VS Code	Abre shared/thresholds.ts.	<code>apply 0.7 · flag 0.5 · needsHuman 0.5</code>	"Estos números son míos, no del modelo. Jev da probabilidades; mi código decide."
3	Claude Code	<code>/triage Pagué con débito y me llegaron dos cargos por el mismo pedido</code>	Claude crea KAN-251 con el MCP y muestra la tabla: bug / crítica / pagos → apply, en ~2 s.	"Claude escribe el ticket. Jev lo clasifica en milisegundos."
4	Jira	Abre KAN-251, el ticket recién creado.	Tipo Error, prioridad Highest, etiqueta jev y un comentario de Jev con cada decisión y su %.	"Cada decisión viene con su probabilidad: se puede auditar."
5	Terminal	<code>npm run batch -- --no-create</code>	22 filas en ~2 s. Acciones: ~16 apply · ~3 apply_and_flag · 3 escalate. Costo de una fracción de centavo.	"22 tickets en dos segundos por menos de un décimo de centavo."
6	Jira	Filtra <code>labels = escalado</code> .	3 tickets: KAN-248 "No funciona", KAN-249 "Cambiar el color del botón de pagar y arreglar...", KAN-250 "¿Esto es normal?".	"Lo que Jev no sabe, no lo inventa: se lo pasa a una persona."

Qué se le pregunta a Jev (4 preguntas por ticket, una sola llamada)

Archivo: `jira-triage/classify.ts`. El state que recibe Jev es { `title`, `description`, `reporter` }. Las instrucciones de prioridad están en inglés porque ahí Jev rinde mejor; las afinamos en el ensayo.

tipo choice	¿Qué tipo de issue es?
bug	Algo que antes funcionaba y ahora falla
feature	Funcionalidad nueva solicitada
deuda_tecnica	Refactor, limpieza, dependencias, pruebas automatizadas, cumplimiento o mejora interna sin cambio visible para el cliente
soporte	Pregunta o duda de uso, sin cambio de código
prioridad choice	What priority does this ticket deserve? Judge the business impact today, not how alarming the wording sounds.

critica	Money or orders are wrong or blocked right now in production: payments fail for a whole card brand, customers are charged twice, discounts or orders are duplicated. Must be fixed today.
alta	Important but not losing money right now: a key flow broken for one segment (e.g. login on one browser, sessions expiring), security vulnerabilities, legal or tax obligations, or work that protects or unlocks revenue (new payment methods, test coverage of checkout).
media	Annoying but with a workaround or limited reach: slow reports, layout glitches on one device, search or export edge cases, delayed notifications, internal refactors, search that misses some results, new dashboards, or new customer-facing pages and buttons that are useful but not urgent.
baja	Cosmetic, a how-to question, or a nice-to-have with little impact: dark mode, slightly blurry images on some screens, questions about how to use the product.

equipo choice	¿Qué equipo debe atenderlo?
pagos	Checkout, pasarelas, tarjetas, cupones, facturación, reembolsos
plataforma	Infra, autenticación, sesiones, APIs internas, colas, notificaciones, rendimiento
frontend	UI web o móvil, estilos, imágenes, navegación, páginas o botones nuevos que ve el cliente
datos	Reportes, métricas, búsqueda, exportaciones, pipelines

needs_human noul	¿El ticket es demasiado ambiguo o contradictorio para clasificarlo sin una persona?
true	Falta información clave, está vacío o pide dos cosas distintas
false	Tiene suficiente detalle para decidir

La petición completa a Jev (ticket T01, real)

Una sola llamada por ticket con las 4 preguntas. Aquí va la primera completa; las otras van igual, con los textos de las tablas de arriba.

```
{
  "model": "jev-1.13.0",
  "state": {
    "title": "El checkout falla con tarjetas Amex en producción",
    "description": "Desde ayer los clientes con American Express ven un error 500 al confirmar el pago. Visa y Mastercard funcionan normal. Afecta a todos los usuarios con Amex.",
    "reporter": "soporte"
  },
  "questions": {
    "tipo": {
      "type": "choice",
      "instructions": "¿Qué tipo de issue es?",
      "criteria": {
        "bug": "Algo que antes funcionaba y ahora falla",
        "feature": "Funcionalidad nueva solicitada",
        "deuda_tecnica": "Refactor, limpieza, dependencias, pruebas automatizadas, ...",
        "soporte": "Pregunta o duda de uso, sin cambio de código"
      }
    },
    "prioridad": "{ type: choice, instructions: \"What priority...\", criteria: {critica, alta, media, baja}"
  },
  "equipo": "{ type: choice, ... }",
  "needs_human": "{ type: noul, ... }"
}
```

La respuesta real de Jev

```
{
  "answers": {
    "tipo": {"type": "choice", "choice": "bug", "confidence": 1, "probabilities": {"deuda_tecnica": 0, "bug": 1, "feature": 0, "soporte": 0}},
    "prioridad": {"type": "choice", "choice": "critica", "confidence": 0.99, "probabilities": {"alta": 0.01, "baja": 0, "critica": 0.99, "media": 0}},
    "equipo": {"type": "choice", "choice": "pagos", "confidence": 1, "probabilities": {"frontend": 0, "pagos": 1, "datos": 0, "plataforma": 0}},
    "needs_human": {"type": "noul", "noul": 0.17}
  }
}
```

Lectura: bug 100 %, crítica 99 %, pagos 100 % y 17 % de que necesite humano. La confianza mínima es $0.99 \geq 0.70$ y el humano $0.17 < 0.50$, así que tu código aplica directo (apply).

Por qué cada pregunta

Pregunta	Para qué sirve	Por qué está redactada así	Qué hace tu código con la respuesta
tipo	Que el ticket entre al tablero con el tipo correcto (Error, Historia, Tarea).	Cuatro opciones excluyentes con una definición operativa cada una ("antes funcionaba y ahora falla"), no etiquetas sueltas.	Lo mapea a Error / Historia / Tarea y lo escribe en Jira si la acción no es escalate.
prioridad	Ordenar el backlog por impacto real, que es lo que hace un manager cada lunes.	En inglés y con ejemplos concretos por nivel. La primera versión, vaga y en español, daba "alta" a casi todo con poca confianza y escalaba 15 tickets.	Highest / High / Medium / Low en Jira. Su confianza suele ser la más baja de las tres, así que es la que más decide la acción.
equipo	Saber a quién le toca, y asignarlo al líder del equipo si está configurado.	Cada equipo descrito por los temas que atiende; tras el ensayo se agregó "páginas o botones nuevos" a frontend.	Va en el comentario de Jev (KAN no tiene campo de equipo) y elige el líder en LEAD_*.
needs_human	Detectar tickets que ninguna clasificación arregla: vacíos, contradictorios o que piden dos cosas.	Es noul (sí/no) porque es una alarma, no una categoría; las opciones dicen qué cuenta como ambiguo.	Si ≥ 0.50 escala aunque las otras confianzas sean altas: es el freno de mano.

Qué decide tu código

En `shared/thresholds.ts`, con la confianza mínima de las tres respuestas (tipo, prioridad, equipo):

```
si needs_human ≥ 0.50      → escalate      (no aplica nada; etiqueta "escalado")
si confianza mínima ≥ 0.70  → apply          (escribe tipo, prioridad y comentario)
si confianza mínima ≥ 0.50  → apply_and_flag (escribe y etiqueta "revisar")
si no                      → escalate      (etiqueta "escalado")
```

Mapeo a Jira (proyecto en español), en `jira-triage/jira.ts`: bug → Error, feature → Historia, deuda técnica y soporte → Tarea; crítica → Highest, alta → High, media → Medium, baja → Low. El equipo va en el comentario (KAN no tiene un campo "Equipo" de lista).

Qué le pide el skill /triage a Claude

Archivo: `.claude/skills/triage/SKILL.md`. Resumen de las instrucciones:

- Del texto del usuario, sacar uno o varios issues (uno por problema), con resumen corto en español y la queja del cliente en la descripción, sin inventar datos.
- Crearlos con el MCP de Atlassian en KAN, tipo Tarea, **sin prioridad ni etiquetas**: la clasificación la pone Jev, no Claude. Nunca con la etiqueta batch.
- Correr `npm run triage -- <KEY>` y no clasificar por su cuenta; si el script falla, mostrar el error y detenerse.

- Mostrar la tabla tal cual, una línea por issue con la decisión, y cerrar con el tiempo y el costo de Jev.

El antes: el mismo triage con Sonnet

npm run batch -- --mode=direct es lo que se hace hoy con IA: un prompt a Sonnet por ticket que pide la clasificación en JSON. Lee los mismos 22 issues reales de KAN (solo lectura, no escribe nada en Jira) y se corre una vez antes de la sesión; queda en results/triage-direct.json.

- El prompt** (directTicketPrompt en jira-triage/classify.ts) se arma con los mismos textos que las preguntas a Jev: las cuatro instrucciones y cada opción con su descripción, sacadas de TICKET_QUESTIONS. Así la comparación es justa: mismas reglas, distinto modelo.
- La respuesta** se valida en parseDirectTicket: si el JSON no viene o trae una opción que no existe, es error y el ticket se escala. Jev nunca tiene ese problema: siempre elige una de las opciones dadas.
- Lo que decide tu código:** Sonnet no da probabilidades, así que no hay umbral que aplicar. Solo quedan dos acciones, aplicar o escalar si el propio LLM dice que necesita humano. No existe "aplicar y marcar para revisión".

```
Clasifica este ticket de soporte de una tienda en línea. Responde SOLO con un objeto JSON, sin texto antes ni después:
{"tipo": "...", "prioridad": "...", "equipo": "...", "needs_human": true|false}

tipo - ¿Qué tipo de issue es?
  - bug: Algo que antes funcionaba y ahora falla
  - feature: ... (las mismas opciones de las tablas de arriba)
prioridad - What priority does this ticket deserve? ...
equipo - ¿Qué equipo debe atenderlo? ...
needs_human - ¿El ticket es demasiado ambiguo o contradictorio ...?

Ticket:
Título: <summary del issue en Jira>
Descripción: <description del issue en Jira>
```

	Antes: Sonnet	Después: Jev	Qué dices
Costo (22 tickets)	\$0.078	\$0.0009	"Los dos aciertan igual. Jev cuesta 87 veces menos, tarda 14 veces menos y además dice qué tan seguro está."
Tiempo	20 s	1.4 s	
Tipo y equipo correctos	19/19	19/19	
Acciones	19 apply · 3 escalate	16 apply · 2 apply_and_flag · 4 escalate	

Si preguntan "¿entonces Sonnet es igual de bueno?": sí, en precisión. La diferencia es costo, tiempo y que Jev da una probabilidad por decisión, que es lo que permite a tu código distinguir "aplico", "aplico y que alguien revise" y "esto es para una persona".

Resultados del ensayo (7 de octubre, servicios reales)

Corrida	Resultado
/triage (KAN-128, cobro duplicado)	bug 89 % · crítica 99 % · pagos 100 % · humano 17 % → apply . Jev en 210 ms, 956 tokens.
batch --no-create (22 tickets)	16 apply · 3 apply_and_flag · 3 escalate (justo T28, T29, T30). Tipo y equipo 19/19 contra lo esperado. 1.8 s, 20,541 tokens, \$0.0021.

Si algo falla

- /triage no crea el issue:** el MCP no está autenticado → /mcp → atlassian. Plan B: crea el issue a mano en Jira y corre npm run triage -- KAN-251 (o la key que te dé Jira).

- **Jira no responde:** `JIRA MOCK=1 npm run batch` clasifica con Jev real sin tocar Jira (crea keys falsas).
- **Sin red:** `npm run batch -- --mock` (Jev y Jira simulados; se ve en amarillo como MOCK).

DEMO 2

Pre-vuelo sobre Medusa real

Minutos 10–18 · Antes de tocar código para un ticket, ¿qué problemas ya hay ahí?

Qué se cuenta

El ticket T03 de la demo 1 (KAN-223, "Reembolso parcial se registra como total") obliga a tocar el código de pagos de **Medusa**, un e-commerce open source real. Son dos archivos de producción con 58 funciones. Mandarlas todas a Claude sería caro y lento. Jev las revisa todas en paralelo en ~2 segundos, marca solo las sospechosas y Claude explica únicamente esas. En el ensayo, la que Jev marcó resultó ser un bug latente real.

Secuencia de pasos

#	Dónde	Qué haces / escribes	Qué debe aparecer	Qué dices
1	Jira	Abre KAN-223 , "Reembolso parcial se registra como total" (T03).	Ya clasificado en la demo 1: bug, pagos.	<i>"Este ticket me toca. Antes de tocar el código, ¿qué hay ahí?"</i>
2	VS Code	Abre <code>medusa-fork/packages/modules/payment/src/services/payment-module.ts</code> .	1,468 líneas de código real.	<i>"Esto es Medusa, un e-commerce open source real. No sembramos nada."</i>
3	Terminal	<code>npm run scan -- --ticket=T03</code>	58 funciones en ~1 s. 1 o 2 marcadas por <code>traga_errores</code> (siempre <code>roundToCurrencyPrecision</code>). ~\$0.003.	<i>"Jev revisó 58 funciones en un segundo y señaló dónde mirar."</i>
4	Terminal o Claude Code	<code>npm run scan -- --ticket=T03 --explain o /pre-vuelo T03</code>	La misma tabla (reutiliza a Jev) y la explicación de Claude solo de lo marcado, en ~7 s.	<i>"Claude solo leyó lo que Jev marcó. Las demás no gastaron un token."</i>
5	VS Code	Ve a <code>roundToCurrencyPrecision</code> (línea 161 de <code>payment-module.ts</code>).	Un <code>try</code> con <code>split(".", "")[1]</code> y un <code>catch {}</code> vacío.	<i>"Un catch vacío: si algo falla, nadie se enteró."</i>

#	Dónde	Qué haces / escribes	Qué debe aparecer	Qué dices
6	Terminal	Pega el node -e de JPY (abajo en esta sección).	USD 2 decimales · JPY → TypeError (el catch de Medusa lo esconde).	"Con yenes, el monto se queda sin redondear y el sistema dice que todo bien."
7	Terminal	<code>npm run costs -- --demo=1,2</code>	Demo 1 y demo 2: unos 2 centavos en total.	"Todo lo que llevamos costó dos centavos."

Los archivos que escanea

Archivo (dentro de medusa-fork)	Funciones
<code>packages/modules/payment/src/services/payment-module.ts</code>	34 (el servicio de pagos)
<code>packages/modules/providers/payment-stripe/src/core/stripe-base.ts</code>	24 (el proveedor de Stripe)

La ruta sale de `MEDUSA_PATH` en `.env` (por defecto `../medusa-fork`). Si falta el clon, el comando dice cómo clonarlo.

Qué se le pregunta a Jev (6 preguntas por función, una sola llamada)

Archivo: `code-health/scan.ts`. El state lleva: archivo, nombre de la función, líneas, parámetros, señales baratas (awaits, catch sospechoso, literales...), las funciones parecidas (`similar_to`) y el código fuente (hasta 6,000 caracteres). Una señal cuenta como marca si su probabilidad es $\geq$ `SCAN_THRESHOLD` (0.60).

responsabilidades choice	How many unrelated responsibilities does this function have? (single responsibility principle) Judge reasons to change, not number of steps.
una	One job, even with several steps: validating input, calling one service or repository, and mapping or returning the result is ONE responsibility (orchestration). Thin wrappers, delegations to a provider and transactional wrappers also count as one.
varias	Mixes concerns that change for different reasons in the same body: e.g. computes money AND sends an email AND writes audit logs or metrics AND changes order status, or business rules mixed with formatting of notifications.
traga_errores noul	¿Captura un error y lo oculta devolviendo éxito o un valor inventado, en vez de propagarlo o reportarlo?
true	Hay un catch que devuelve ok/true, un id falso o un valor por defecto como si nada hubiera pasado
false	Los errores se propagan, se devuelven como error o se registran de forma explícita
muta_entrada noul	¿Modifica directamente un objeto o arreglo que recibió como parámetro?
true	Asigna propiedades o hace push sobre un parámetro; quien llamó ve el cambio
false	Trabaja sobre copias o devuelve valores nuevos
numero_magico noul	¿Usa números literales con significado de negocio sin nombre (tasas, umbrales, porcentajes)?

true	Aparecen literales como 0.16, 1.16, 0.9 que representan impuestos, umbrales o reglas de negocio
false	Los números están nombrados como constantes o son triviales (0, 1, 100 para redondear)
logica_duplicada noul	Considerando las funciones similares listadas en similar_to, ¿duplica lógica que ya existe en otra función, con variaciones sutiles?
true	Reimplementa el mismo cálculo o regla de negocio que otra función listada, pero con fórmula, redondeo o condición distinta
false	No hay función similar, o la similitud es estructural: un método público que delega en su versión privada (create/create_), varios métodos que llaman al mismo proveedor con distinto método, o CRUD con la misma forma
severidad score	Si un desarrollador toca esta función para la tarea, ¿qué tan probable es que los problemas detectados causen un bug en producción?
0	Ninguno: la función está limpia
1	Bajo: deuda de estilo, no afecta el comportamiento
2	Medio: puede producir resultados incorrectos en casos borde
3	Alto: produce cobros, estados o montos incorrectos en casos normales

La petición completa a Jev (roundToCurrencyPrecision, real)

Una llamada por función con las 6 preguntas; las 58 funciones se mandan en paralelo. Las señales baratas (signals) las calcula split.ts sin ningún modelo.

```
{
  "model": "jev-1.13.0",
  "state": {
    "file": "packages/modules/payment/src/services/payment-module.ts",
    "function": "roundToCurrencyPrecision",
    "lines": 18,
    "params": [
      "amount",
      "currencyCode"
    ],
    "signals": {
      "lines": 18,
      "awaits": 0,
      "side_effect_kinds": 0,
      "catch_swallow_hint": true,
      "decimal_literals": [
        "0.1111111"
      ],
      "mutated_params": [],
      "max_nesting": 4,
      "comment_sections": 0
    },
    "similar_to": [],
    "source": "  protected roundToCurrencyPrecision(\n    amount: BigNumberInput,\n    currencyCode: string\n  ): BigNumberInput {\n    let precision: number | undefined = undefined\n    try {\n      const formatted = Intl.NumberFormat(undefined, {\n        style: \"currency\", \n        ...
    },
    "questions": {
      "responsabilidades": {
        "type": "choice",
        "instructions": "How many unrelated responsibilities does this function have? ...",
        "criteria": {
          "una": "One job, even with several steps: ...",
          "varias": "Mixes concerns that change for different reasons ..."
        }
      }
    }
  }
```

```

    },
    "traga_errores": "{ type: noul, ... }",
    "muta_entrada": "{ type: noul, ... }",
    "numero_magico": "{ type: noul, ... }",
    "logica_duplicada": "{ type: noul, ... }",
    "severidad": "{ type: score, criteria: [Ninguno, Bajo, Medio, Alto] }"
  }
}

```

La respuesta real de Jev

```

{
  "answers": {
    "responsabilidades": {"type": "choice", "choice": "una", "confidence": 0.92, "probabilities": {"una": 0.96, "varias": 0.04}},
    "traga_errores": {"type": "noul", "noul": 0.84},
    "muta_entrada": {"type": "noul", "noul": 0.07},
    "numero_magico": {"type": "noul", "noul": 0.25},
    "logica_duplicada": {"type": "noul", "noul": 0.05},
    "severidad": {"type": "score", "score": 2.3, "confidence": 0.57, "probabilities": {"0": 0.02, "1": 0.02, "2": 0.59, "3": 0.37}}
  }
}

```

Lectura: una sola responsabilidad (96 %), pero **traga errores 84 %** $\geq 0.60 \rightarrow$ marcada. Severidad 2.3, entre "medio" y "alto". Por eso pasa a Claude.

Por qué cada pregunta

Pregunta	Para qué sirve	Por qué está redactada así	Qué hace tu código con la respuesta
responsabilidades	Detectar funciones que mezclan cosas que cambian por razones distintas: tocar una rompe la otra.	Pide juzgar "razones para cambiar, no número de pasos" y aclara que orquestar (validar $\rightarrow$ llamar $\rightarrow$ devolver) es una sola. Sin eso marcaba 21 de 58 en Medusa.	Marca si sale "varias" con confianza ≥ 0.60 .
traga_errores	El bug más caro y más invisible: el error ocurre y el sistema dice que todo salió bien.	Describe la forma exacta: un catch que devuelve éxito, un id falso o un valor por defecto.	Marca si ≥ 0.60 . Es la que encontró el bug de JPY.
muta_entrada	Efectos secundarios: quien llamó ve su objeto cambiado aunque la operación falle a medias.	Concreta: asignar propiedades o hacer push sobre un parámetro.	Marca si ≥ 0.60 .
numero_magico	Reglas de negocio escondidas en literales (0.16 de IVA, 0.9 de umbral).	Separa los literales de negocio de los triviales (0, 1, 100 para redondear) para no hacer ruido.	Marca si ≥ 0.60 .
logica_duplicada	La misma regla implementada dos veces con variaciones: centavos que no cuadran.	Jev recibe las funciones parecidas (similar_to); se aclara que delegar (create/create_) o CRUD con la misma forma no es duplicar.	Marca si ≥ 0.60 .
severidad	Ordenar lo marcado: qué arreglar antes de empezar la tarea.	Escala de 4 niveles con consecuencias concretas ("cobros o montos incorrectos").	Ordena la tabla y colorea la columna Sev.; no decide si se marca.

El prompt que recibe Claude (solo para las funciones marcadas)

Archivo: code-health/explain.ts · modelo claude-sonnet-5-5 · máximo 500 tokens de salida.

Eres un revisor senior. Un modelo de decisiones marcó esta función con: <señales y %>.
Un desarrollador va a tocar este archivo para la tarea: "<título del ticket>".
En español y en máximo 150 palabras: 1) confirma o descarta cada señal con la línea concreta;
2) di qué bug puede causar al hacer la tarea; 3) propón el refactor mínimo (nombres de funciones nuevas si aplica). Sin preámbulos.

Archivo: <ruta>
<código de la función>

Resultados del ensayo

Función	Marca de Jev	Qué dijo Claude
roundToCurrencyPrecision payment-module.ts	traga_errores ~81 % · severidad 2.2 · sale siempre	Confirmada. El catch vacío esconde un TypeError: con monedas sin decimales (JPY) split(".") [1] no existe y el monto queda sin redondear. Se usa al calcular capturas. Bug latente real.
refundPayment stripe-base.ts	traga_errores ~60 % · en el límite (a veces entra, a veces no)	Parcialmente confirmada: se traga a propósito el error "ya reembolsado" y devuelve éxito, sin registro ni aviso.

Totales del ensayo: 58 funciones · Jev ~1.8 s · ~82,000 tokens · \$0.0034 · Claude 1 llamada · ~1,000 tokens de entrada / ~430 de salida.

El antes: Claude revisa las 58 funciones

Sin Jev no sabes qué funciones mirar, así que hay que mandarlas todas al LLM. `npm run scan -- --ticket=T03 --mode=direct` lo hace sobre el mismo código real de Medusa: una llamada a Sonnet por función, con el código completo. Se corre una vez antes de la sesión y queda en `results/code-health-direct.json`.

- **El prompt** (`directReviewPrompt` en `code-health/explain.ts`) pide las mismas cinco señales que se le preguntan a Jev, con las mismas claves, y que la primera línea sea `MARCAS: traga_errores, ... 0 MARCAS: ninguna`. Después, solo si marcó algo, la misma explicación que pide el modo con Jev.
- **La lectura** (`parseMarcas`) solo cuenta claves conocidas: si Claude inventa otra, se ignora. Así se puede contar cuántas marcó cada uno.
- **Lo que cambia:** con Jev, Claude solo lee las 2 funciones marcadas. Sin Jev, Claude lee las 58, y las 43 que resultan limpias también cuestan: hubo que leerlas para saberlo.

Eres un revisor senior. Un desarrollador va a tocar este archivo para la tarea: "Reembolso parcial se registra como total".
Revisa esta función buscando: `varias_responsabilidades (...)`; `traga_errores (...)`; `muta_entrada (...)`; `numero_magico (...)`; `logica_duplicada (...)`.
La primera línea de tu respuesta debe ser exactamente "MARCAS: " seguida de las claves que apliquen separadas por coma, o "MARCAS: ninguna".
Después, solo si marcaste algo, en español y en máximo 150 palabras: 1) confirma cada señal con la línea concreta; 2) di qué bug puede causar al hacer la tarea; 3) propón el refactor mínimo. Sin preámbulos.

Archivo: <ruta>
<código de la función>

	Antes: Sonnet lee todo	Después: Jev filtra, Claude explica	Qué dices
Llamadas	58 a Sonnet	58 a Jev + 2 a Sonnet	<i>"Claude marcó 15 y Jev 2. Las 2 de Jev están entre las 15, y una es el bug de JPY. Lo demás es ruido que alguien tendría que revisar."</i>
Costo	\$0.235	\$0.022	
Tiempo	42 s	~1 s Jev + ~11 s explicación	
Funciones marcadas	15 (entre ellas roundToCurrencyPrecision y refundPayment)	2: roundToCurrencyPrecision y refundPayment	

Honestidad: sin la lista de funciones parecidas que recibe Jev (`similar_to`), Claude marca "lógica duplicada" con más facilidad. Varias de sus 15 marcas son discutibles, no necesariamente falsas.

El bug, en vivo (minuto 15)

Es la misma lógica que usa Medusa en `roundToCurrencyPrecision`, en una línea:

```
node -e 'for (const c of ["USD","JPY"]) { try { console.log(c, Intl.NumberFormat(undefined, {style:"currency",currency:c}).format(0.111111).split(".")[1].length, "decimales") } catch (e) { console.log(c, "→", e.constructor.name, "(el catch de Medusa lo esconde)") } }'

USD 2 decimales
JPY → TypeError (el catch de Medusa lo esconde)
```

Qué decir si preguntan "¿y las demás?" En el ensayo, Jev con criterios vagos marcaba 21 de 58 por "varias responsabilidades" y Claude descartaba casi todas. Se afinaron las descripciones de las opciones (no los umbrales) para que la orquestación normal cuente como una responsabilidad. Eso es justo el punto: las reglas de negocio viven en las opciones y en tu código, y se ajustan como cualquier código.

Si algo falla

- **Sin red:** `npm run scan -- --ticket=T03 --explain --mock` corre sobre Medusa con Jev y Claude simulados (necesita `../medusa-fork`). Las marcas del mock son heurísticas: no prometas `refundPayment`.
- **Respaldo de la corrida real:** `results-backup/prevuelo-T03-medusa.txt` (la salida completa con la explicación de Claude).
- **Claude no responde:** corre sin `--explain` (solo Jev) y lee la explicación del respaldo.

DEMO 3

Router de modelos para revisar PRs

Minutos 18–26 · ¿Cada PR merece al modelo grande, al pequeño o a un checklist sin LLM?

Qué se cuenta

Un equipo revisa PRs con un LLM. La costumbre es mandarlo todo al modelo grande. Aquí hay 20 PRs reales de Medusa recreadas como PRs abiertas en el fork de LIDR. El "antes" (todo a Sonnet) ya está calculado. En vivo, Jev decide la ruta de cada PR en una llamada: reglas locales sin LLM, Haiku o Sonnet. Lo que toca dinero va siempre al modelo grande. Al final se comparan tokens, costo y la calidad de la revisión de la PR de pagos.

Secuencia de pasos

#	Dónde	Qué haces / escribes	Qué debe aparecer	Qué dices
1	Navegador	Abre <code>github.com/LIDR-academy/medusa/pulls</code> .	20 PRs abiertas, sin comentarios.	"20 PRs reales de Medusa esperando revisión."
2	Terminal	<code>npm run costs</code>	La fila "Demo 3 · revisión de PRs (direct)": 20 a Sonnet, 33,262 / 9,893 tokens, ~\$0.17.	"Así se hace hoy: todo al modelo grande. Esto ya lo corrimos."
3	VS Code	Abre <code>llm-router/router.ts</code> y luego <code>routeFor</code> en <code>shared/thresholds.ts</code> .	Dos preguntas a Jev; tres caminos: reglas, Haiku, Sonnet.	"Una pregunta, tres caminos. Lo que toca dinero siempre va al grande."
4	Terminal	<code>npm run review -- --mode=jev --data=data/prs-fork.json --comment</code>	Una línea por PR con su ruta y confianza; ~40 s en total.	Mientras corre: "Jev decide en milisegundos; lo lento es la revisión."
5	Navegador	Refresca una PR, por ejemplo la 15 o la 12.	Comentario "Router con Jev" con complejidad, dinero y ruta.	"La decisión queda escrita en la PR, con su porqué."
6	Terminal	<code>npm run compare</code>	Tabla direct frente a jev; ahorro cercano a ~50 % y proyección a 500 PRs al mes.	Silencio. Luego el número.
7	Terminal	<code>npm run show-review -- PR-12</code>	La PR de pagos con Sonnet en ambos modos; las dos revisiones ven los mismos riesgos.	"Donde importaba, la calidad es la misma."

Qué se le pregunta a Jev (2 preguntas por PR, una sola llamada)

Archivo: `llm-router/router.ts`. El state lleva: título, hasta 25 archivos, número de archivos, adiciones, eliminaciones, descripción (600 caracteres) y resumen del diff (3,000).

<code>complejidad</code> choice	¿Qué tan complejo es revisar este cambio con criterio de un revisor senior?
<code>trivial</code>	Tipos, docs, formato, renombrar, bumps de versión rutinarios, cambios sin lógica
<code>media</code>	Refactor acotado, endpoint nuevo, lógica simple, pruebas, i18n con lógica
<code>alta</code>	Pagos, reembolsos, créditos, auth/sesiones, migraciones, concurrencia, colas, seguridad, cambios que tocan muchos paquetes
<code>touches_money</code> noul	¿El cambio toca cobros, pagos, reembolsos, créditos, gift cards o datos financieros?
<code>true</code>	Modifica flujos o registros de dinero

false

No toca dinero

La petición completa a Jev (PR-15, real)

Una llamada por PR con las 2 preguntas, aquí completas. La descripción y el diff van recortados en esta hoja (en la llamada real, hasta 600 y 3,000 caracteres).

```
{
  "model": "jev-1.13.0",
  "state": {
    "title": "fix(loyalty-plugin): correct transaction note for store credit debits",
    "files": [
      ".changeset/store-credit-debit-note.md",
      "packages/plugins/loyalty/src/workflows/carts/workflows/confirm-cart-credit-lines.ts"
    ],
    "changed_files": 2,
    "additions": 9,
    "deletions": 1,
    "description": "Recreada para la demo de Jev a partir de **medusajs/medusa#17077**\n(https://github.com/medusajs/medusa/pull/17077).\n\nCommit original:\n`2e6554c59e1d95b9ac52329d40190ce9d6ed1dd7` · base: `demo-base` (ch...",
    "diff_summary": "### .changeset/store-credit-debit-note.md\n@@ -0,0 +1,5 @@\n+---\n+\"@medusajs/loyalty-plugin\": patch\n+---\n+\"fix(loyalty-plugin): correct transaction note for store credit debits\"\n+###\n+packages/plugins/loyalty/src/workflows/carts/workflows/confirm-cart-credit-lines.ts\n@@ -199,7 +199,10 @@\n+export const co..."
  },
  "questions": {
    "complejidad": {
      "type": "choice",
      "instructions": "¿Qué tan complejo es revisar este cambio con criterio de un revisor senior?",
      "criteria": {
        "trivial": "Typos, docs, formato, renombrar, bumps de versión rutinarios, cambios sin lógica",
        "media": "Refactor acotado, endpoint nuevo, lógica simple, pruebas, il8n con lógica",
        "alta": "Pagos, reembolsos, créditos, auth/sesiones, migraciones, concurrencia, colas, seguridad, ..."
      }
    },
    "touches_money": {
      "type": "noul",
      "instructions": "¿El cambio toca cobros, pagos, reembolsos, créditos, gift cards o datos financieros?",
      "criteria": {
        "true": "Modifica flujos o registros de dinero",
        "false": "No toca dinero"
      }
    }
  }
}
```

La respuesta real de Jev

```
{
  "answers": {
    "complejidad": {
      "type": "choice",
      "choice": "alta",
      "confidence": 0.06,
      "probabilities": {
        "trivial": 0.28,
        "alta": 0.37,
        "media": 0.35
      }
    },
    "touches_money": {
      "type": "noul",
      "noul": 0.97
    }
  }
}
```

Lectura: la complejidad está repartida (alta 37 %, media 35 %, trivial 28 %; confianza 6 %), pero **toca dinero 97 %**. La primera regla de tu código gana: va a Sonnet por "toca dinero". Un cambio de 9 líneas, pero en créditos de tienda.

Por qué cada pregunta

Pregunta	Para qué sirve	Por qué está redactada así	Qué hace tu código con la respuesta
complejidad	Decidir cuánto modelo merece la revisión: reglas sin LLM, Haiku o Sonnet.	Tres niveles descritos por el tipo de cambio que revisaría un senior, no por el tamaño del diff.	trivial → rules, media → Haiku, alta → Sonnet; si la confianza es < 0.60, Sonnet por si acaso.
touches_money	Red de seguridad: lo que toca dinero nunca se revisa con el modelo barato.	Es noul (sí/no) y lista los dominios de dinero (cobros, reembolsos, créditos, gift cards).	Si ≥ 0.50 va a Sonnet sin importar la complejidad. Se evalúa primero.

Qué decide tu código

En `shared/thresholds.ts`, función `routeFor`, en este orden:

```
si toca dinero ≥ 0.50      → sonnet  ("toca dinero")
si confianza < 0.60        → sonnet  ("confianza baja": ante la duda, el modelo grande)
si complejidad = trivial   → rules   (checklist local, cero tokens)
si complejidad = media     → haiku
si complejidad = alta      → sonnet
```

La ruta **rules** no llama a ningún LLM: revisa que haya descripción, que el cambio sea de 100 líneas o menos, que no toque archivos sensibles (`payment`, `refund`, `auth`, `session`, `migration`, `.env`, `secret`) y que no agregue `console.log` ni `TODO/FIXME` (`rulesReview` en `llm-router/llm.ts`).

El prompt de revisión (idéntico en `direct` y en `jev`)

El "antes" de esta demo es `npm run review -- --mode=direct --data=data/prs-fork.json`: las 20 PRs reales del fork van a Sonnet con este mismo prompt. Es lo que guarda `results/direct.json`.

Archivo: `llm-router/llm.ts` · Haiku 4.5 o Sonnet 5.5 según la ruta · máximo 600 tokens de salida.

```
Eres un revisor de código senior. Revisa esta pull request y responde en español,
en máximo 180 palabras:
1) Riesgo principal (o "ninguno relevante").
2) Hasta 3 observaciones concretas con archivo.
3) Veredicto: aprobar / aprobar con cambios / pedir cambios.

Título · Archivos · +adiciones / -eliminaciones · Descripción (800) · Diff (recortado)
```

Resultados del ensayo

Modo	Rutas	Tokens LLM (entrada / salida)	Tiempo
direct (ya calculado)	20 a Sonnet	33,262 / 9,893	38 s
jev (ensayo completo, con --comment)	8 Sonnet · 2 Haiku · 10 reglas	16,050 / 5,936 + 23,661 de Jev (\$0.0010)	40 s

Proyección que imprime `npm run compare` para 500 PRs al mes con esta mezcla: `direct` \$4.14 → `jev` \$2.01, **-51 %** (en otra corrida, -48 %). Ejemplos de escalado: PR-15 y PR-6 (`loyalty-plugin`, toca dinero 97 % y 93 %) a Sonnet; PR-16 (i18n checo, confianza baja 28 %) a Sonnet. Di el número que salga en vivo.

Cierre técnico (minuto 26)

`npm run review -- --mode=jev --data=data/prs-fast-jev-compaction.json --out=results/cierre.json --concurrency=6` (~30 s): el mismo router revisa PRs de *fast-jev-compaction*, un plugin de Claude Code que a su vez usa Jev. Mismo patrón en otro dominio.

Si algo falla

- **Sin red:** `npm run review -- --mode=jev --mock y npm run compare` (se ve como MOCK).
- **GitHub no deja comentar:** corre sin `--comment`; la terminal muestra igual la ruta de cada PR.
- **Se perdió direct.json:** `npm run review -- --mode=direct --data=data/prs-fork.json` (~40 s, no en vivo).

7. Cierre y costos

`npm run costs` lee todo lo que hay en `results/` y arma una tabla por ejecución: unidades, llamadas y tokens de Jev, llamadas y tokens de LLM, dólares y tiempo. `/costos` hace lo mismo y lo explica en una línea por demo. Precios en `llm-router/prices.json` (verificados el 6 de octubre de 2026):

Modelo	Entrada / millón	Salida / millón
Jev (jev-1.13.0)	\$0.042	gratis
Claude Sonnet 5.5	\$2.00	\$10.00
Claude Haiku 4.5	\$1.00	\$5.00

Referencia del ensayo: demo 1 + demo 2 con Jev costaron unos 2 centavos, casi todo de Claude; Jev, una fracción de centavo. Las mismas dos demos sin Jev (todo a Sonnet) costaron 31 centavos. La guía pide que la demo completa con Jev quede bajo \$0.10 sin contar los "antes" precalculados.

Demo	Antes (todo a Sonnet)	Después (Jev decide)	Ahorro
1 · Triage (22 tickets)	\$0.078	\$0.0009	~87× más barato
2 · Pre-vuelo (58 funciones)	\$0.235	\$0.022	~10× más barato
3 · Router (20 PRs)	\$0.166	\$0.077	-53 %

Secuencia del cierre

#	Dónde	Qué haces / escribes	Qué debe aparecer	Qué dices
1	Terminal	<code>npm run review -- --mode=jev --data=data/prs-fast-jev-compaction.json --out=results/cierre.json --concurrency=6</code>	12 PRs del plugin con su ruta; ~30 s.	<i>Mientras corre: "Este plugin de Claude Code usa Jev. Lo revisa un router que usa Jev."</i>
2	Diapositiva	El patrón: estado → pregunta tipada → Jev → probabilidad → tu código decide → acción.		<i>"Jev no escribe, Jev elige. Lo que elige lo ejecuta tu código."</i>
3	Terminal o Claude Code	<code>npm run costs 0 /costos</code>	Cada demo con su fila antes (direct) y después (jev), y el cierre en su propia fila. Total ~\$0.67, de los que ~\$0.48 son los tres "antes" precalculados.	<i>"Esto costó toda la demo."</i>

Frases para el cierre

- "Jev no escribe, Jev elige. Lo que elige lo ejecuta tu código."
- "En la demo 2 nadie sabía que ese bug estaba en Medusa. Jev señaló una función de 58 y ahí estaba."
- "Claude solo trabajó donde hacía falta, con el modelo que hacía falta."

Límites que conviene decir en voz alta

- Jev no explica ni razona: no sustituye la revisión de código ni el análisis de un bug. En la demo 2, Jev dice dónde mirar; Claude dice qué está mal.

- Como cualquier modelo, es vulnerable a texto malicioso dentro del ticket o la PR. Por eso los umbrales y la revisión humana viven en tu código.
- El ahorro depende de la mezcla de PRs. Di el número que salga.

8. Referencia rápida de comandos y skills

Comando / skill	Qué hace	Demo
/triage <texto>	Claude crea el issue con el MCP y llama a npm run triage	1
npm run triage -- KAN-251	Jev clasifica un issue que ya existe y escribe en Jira	1
npm run batch -- --no-create	Clasifica todos los tickets con etiqueta batch sin terminar	1
npm run batch -- --mode=direct	El antes: Sonnet clasifica los mismos issues de KAN (solo lectura)	prep
npm run seed -- --done=8	Siembra los 30 tickets y pasa 8 a Finalizada (con keys nuevas)	prep
npm run reset	Borra los issues de la demo en KAN y vacía results/, salvo los tres "antes"	prep
npm run reset -- --baselines	Además borra los tres "antes"	prep
npm run scan -- --ticket=T03	Pre-vuelo con Jev sobre Medusa	2
... --explain · /pre-vuelo T03	Además Claude explica lo marcado	2
npm run scan -- --ticket=T03 --mode=direct	El antes: Claude revisa las 58 funciones, sin Jev	prep
npm run review -- --mode=direct jev	Revisión de PRs todo a Sonnet o con el router	3
--data=data/prs-fork.json --comment	Usa las PRs del fork y comenta en GitHub	3
npm run compare	Tabla direct frente a jev y proyección	3
npm run show-review -- PR-12	Revisión de una PR en ambos modos	3
npm run costs · /costos	Costo de cada ejecución (el cierre al final)	todas
npm run costs -- --demo=1,2	Solo las demos indicadas (minuto 17)	2
--out=results/cierre.json	En review: guarda aparte para no pisar results/jev.json	cierre
--fresh	En scan: vuelve a preguntarle a Jev en lugar de reutilizar	2
--mock	En cualquier comando: sin Jev, Claude ni Jira (ensayo sin red)	todas
npm test	14 pruebas: umbrales y lectura de las respuestas del LLM	prep

9. Qué se ajustó en el ensayo (por si preguntan)

- **Jev rechazaba todas las llamadas reales (400):** el cliente mandaba un campo `user` que la API no acepta. Se quitó en `shared/jev.ts`.
- **El triage escribía en issues equivocados:** emparejaba por posición y los 8 en Finalizada descuadraban el orden. Ahora empareja por título (`jira-triage/batch.ts`).
- **Prioridad:** con criterios vagos Jev daba "alta" a casi todo y escalaba 15 tickets; con criterios concretos en inglés escalan exactamente los 3 ambiguos.
- **Pre-vuelo en Medusa:** de 21 marcas de ruido a 1–2 marcas útiles afinando las opciones de responsabilidades y duplicación. La referencia para no romperlo es `results-backup/code-health-T03-medusa.json` (sample-app ya no

está en el repo).

- **Claude por CLI:** la API de Anthropic no tiene saldo; `CLAUDE_ENGINE=cli` usa `claude -p` con tu sesión. Sin herramientas ni MCP, cada llamada entra con ~1,000 tokens, así que los costos siguen limpios.
- **Demo 1 en vivo sin webhook:** en lugar de túnel y webhook, el skill `/triage` crea el issue con el MCP y lo clasifica.
- **Un "antes" para cada demo:** las demos 1 y 2 solo medían a Jev. Ahora `batch` y `scan` tienen `--mode=direct` (todo a Sonnet, sobre los mismos datos reales), como ya lo tenía la demo 3. `npm run reset` los conserva porque se precaculan una vez.
- **Lo que encontró el ensayo completo:** ver la sección 10.

10. Ensayo completo del 7 de octubre

Se corrió la demo entera en orden, con servicios reales: Jira, Jev, Claude por la CLI y GitHub (incluidos los 20 comentarios en las PRs). Luego se corrigió lo que falló, se volvió a probar y se dejó todo limpio.

Tiempos reales de cada comando

Min	Comando	Tiempo real	Resultado
3	/triage (MCP + npm run triage)	~2 s el script	KAN-159 → Error · Highest · pagos · apply; comentario en Jira
6	npm run batch -- --no-create	2.4 s	16 apply · 3 apply_and_flag · 3 escalate · 19/19
11	npm run scan -- --ticket=T03	1.0 s	58 funciones · 1 o 2 marcadas
13	... --explain	6.6 s (antes 18 s)	Mismas marcas que el min 11; Claude explica en paralelo
15	node -e de JPY	instantáneo	USD 2 decimales · JPY → TypeError
17	npm run costs -- --demo=1,2	instantáneo	Demo 1 + demo 2: unos 2 centavos
20	review --mode=jev ... --comment	40 s	8 Sonnet · 2 Haiku · 10 reglas; 20 comentarios en GitHub
23	npm run compare	instantáneo	~51 % de costo frente a todo a Sonnet
25	npm run show-review -- PR-12	0.2 s	Sonnet en ambos modos; las dos revisiones ven los mismos riesgos
26	review ... fast-jev-compaction -- out ... --concurrency=6	31 s (antes 60 s)	12 PRs: 2 reglas · 8 Haiku · 2 Sonnet
29	npm run costs	instantáneo	Total ~\$0.35, de los que \$0.17 son el direct precalculado

Qué falló y cómo se corrigió

Problema en el ensayo	Por qué importaba en vivo	Corrección
El filtro labels = revisar mostraba 6 tickets, no 3	Ibas a decir "los 3 ambiguos" y se veían 6: los aplicados con marca también llevaban revisar	Los escalados llevan su propia etiqueta, escalado. El filtro del minuto 8 es labels = escalado.
El minuto 11 dijo "1 marcada" y el 13 "2 marcadas"	--explain volvía a preguntarle a Jev, y refundPayment está en el límite del umbral	El pre-vuelo guarda las respuestas de Jev; --explain las reutiliza. Mismos números en los dos pasos.
--explain tardaba 18 s	Silencio largo en pantalla	Claude explica las funciones en paralelo: 6.6 s.
npm run costs en el min 17 ya mostraba la demo 3	Revelaba el costo de "todo a Sonnet" antes de presentarlo	npm run costs -- --demo=1,2.
El cierre pisaba results/jev.json y tardaba 60 s	El costs final ya no mostraba la demo 3; un minuto de espera en el cierre	--out=results/cierre.json (fila propia en costs) y --concurrency=6: 31 s.
El costo de un ticket salía como \$0.0000	Parece gratis o un error	Bajo una décima de centavo se muestran 6 decimales: \$0.000039.

Lo que el ensayo no pudo probar

- **Los skills escritos desde Claude Code abierto en jev-dev-demo** (`/triage`, `/pre-vuelo`, `/costos`). En el ensayo se hizo lo mismo paso a paso, porque esta sesión arrancó en otra carpeta. Pruébalos una vez antes de la sesión.
- **La red del lugar y el hotspot.** Todo corrió en tu red. Ensaya una vez con el celular.
- **Tu ritmo hablando.** Los comandos suman unos 2.5 minutos de espera en total; el resto del tiempo es tuyo. Cronométrate una vez con el minuto a minuto.

Estado después del ensayo: KAN con 30 tickets, KAN-221 a KAN-250 (22 por hacer, 8 en Finalizada), 0 comentarios en las PRs del fork y results/ solo con los tres "antes" (`triage-direct.json`, `code-health-direct.json` y `direct.json`). El `/triage` en vivo creará KAN-251. Listo para la sesión.